

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

THE UNIVERSITY OF TEXAS AT ARLINGTON

DETAILED DESIGN SPECIFICATION

CSE 4316 / CSE 4317: SENIOR DESIGN

Spring/Summer 2026



AgriHome

GOVINDA AWASTHI

LAXMAN BHUSHAL

THUC DANG

SAGAR KHADKA

MUHAMMAD SAAD MEMON

ALEX TABLIZO

NATHAN NGUYEN TRAN

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	06.16.2026	SK, TD	Initial DDS and hardware specification draft
0.2	06.17.2026	MM, AT	Layer-level subsystem designs
0.3	06.18.2026	LB, NT	Template formatting, logo, lists
1.0	06.18.2026	GA	Reviewed submission version
2.0	07.26.2026	Team	Comprehensive detailed design revision

CONTENTS

1	AH-109: Introduction and System Overview	7
1.1	Introduction	7
1.2	System Overview	7
2	AH-110: Frontend Layer Subsystems	10
2.1	Layer Purpose	10
2.2	Layer Hardware	10
2.3	Layer Operating System	10
2.4	Layer Software Dependencies	10
2.5	Dashboard Overview Subsystem	11
2.6	Image Feed Display Subsystem	11
2.7	Plant Health Result UI Subsystem	12
2.8	Monitoring History Subsystem	12
2.9	Alerts and Notifications UI Subsystem	12
2.10	Responsive Navigation and Layout Subsystem	12
2.11	Device Management and Capture Control Subsystem	12
3	AH-111: Backend API Layer Subsystems	13
3.1	Layer Purpose	13
3.2	Layer Hardware	13
3.3	Layer Operating System	13
3.4	Layer Software Dependencies	13
3.5	Authentication and User Management API	13
3.6	Plant and Tray Management API	14
3.7	Image Upload and Retrieval API	14
3.8	Edge Device API	14
3.9	Edge Service Mapping	15
3.10	Plant Health Analysis API	15
3.11	Alerts API	15
4	AH-112: Database / Data Layer Subsystems	16
4.1	Layer Purpose	16
4.2	Layer Hardware	16
4.3	Layer Operating System	16
4.4	Layer Software Dependencies	16
4.5	Database Schema Summary	16
4.6	tray_systems Table	17
4.7	plants Table	17
4.8	camera_captures Table	18
4.9	edge_devices Table	18
4.10	edge_device_commands Table	18
4.11	capture_pose_sequences / capture_poses	18

4.12	capture_schedules Table	18
4.13	prediction_results / plant_reports / monitoring_events.....	18
4.14	Indexing, Retention, and Validation Strategy	18
5	AH-113: Storage File Management Systems	19
5.1	Layer Purpose	19
5.2	Layer Hardware	19
5.3	Layer Operating System	19
5.4	Layer Software Dependencies.....	19
5.5	File Management Subsystems.....	19
5.6	Image Upload Subsystem.....	20
5.7	File Naming and Folder Organization Subsystem	20
5.8	Image Retrieval Subsystem	20
5.9	Image Cleanup and Archiving Subsystem.....	20
5.10	Security and Access Control Subsystem	20
6	AH-114: Hardware / Camera Layer Subsystems	21
6.1	Layer Purpose	21
6.2	Layer Hardware	21
6.3	Layer Operating System	21
6.4	Layer Software Dependencies.....	22
6.5	Camera Capture Subsystem.....	22
6.6	Plant Rack Positioning Subsystem.....	22
6.7	Device Communication Subsystem.....	22
6.8	Power and Reliability Subsystem.....	22
6.9	Hardware Layer Limitations	22
7	AH-115: System Overview and Architecture Diagram	23
7.1	Architecture Explanation	23
7.2	Architecture Diagram	23
7.3	Architecture Data Flow	23
7.4	Interface Summary	24
7.5	Design Rationale	24
7.6	Current Implementation Connection	25
8	AH-116: Final Review and Submit	25
8.1	Final Review Checklist.....	25
8.2	Revision History Table	25
8.3	List of Figures / Tables	25
8.4	Appendix Suggestions.....	25
9	Appendix A: Suggested Supporting Material	26
10	Appendix B: Subsystem to Module / Table Map	27
11	References	28

LIST OF FIGURES

1	System Overview	9
2	Capture control UI subsystem: operator requests become device capture commands.	12
3	Camera capture subsystem: target pose becomes an ingest-ready plant image.	22
4	AgriHome runtime detail flow for operator actions, edge captures, storage, and optional analysis.....	23

LIST OF TABLES

1	AgriHome system layers.	8
2	Frontend layer software dependencies.....	10
3	Dashboard Overview subsystem design details	11
4	Image Feed Display subsystem design details	11
5	Edge capabilities mapped to services and records.	15
6	AgriHome relational record groups.	17
7	Storage subsystem summary.	19
8	Hardware prototype component summary.	21
9	Backend API endpoint summary for selected interfaces	24
10	DDS subsystems mapped to implementation areas and records.....	27

1 AH-109: INTRODUCTION AND SYSTEM OVERVIEW

1.1 INTRODUCTION

This Detailed Design Specification describes the internal design of AgriHome for the Spring/Summer 2026 senior-design project. It expands the project requirements and architecture into layer-level and subsystem-level designs for the Vision Console, application services, relational domain records, media storage, Raspberry Pi edge-device communication, rack-mounted XY camera capture hardware, and optional computer-vision processing. The document follows architecture-description and detailed software-design practices at a student-project scale [1, 2].

The document preserves the original DDS organization used by the team: AH-109 introduces the overall design, AH-110 through AH-114 describe detailed layer subsystems, AH-115 provides the system overview and architecture diagram, AH-116 records final-review material, and appendices collect supporting implementation and mapping information. The design is written to explain what each subsystem owns, how it interacts with adjacent subsystems, and what data or hardware responsibilities are assigned to it.

AgriHome is a vision-based plant monitoring system where the camera is attached to a rack or shelf motion system and moves on X-axis and Y-axis paths to capture individual plants. The Raspberry Pi-class edge device coordinates movement, capture, command status, heartbeat reporting, image transfer, and recovery while the application presents the resulting records through the Vision Console.

1.2 SYSTEM OVERVIEW

Operators use the AgriHome Vision Console to view trays, plants, captures, device status, schedules, and plant-health history. Application services validate requests, enforce ownership and device access, persist structured records, and coordinate commands. Media storage holds image files referenced from capture records. A Raspberry Pi edge device receives capture commands and drives the rack-mounted camera carriage to selected plant positions. Optional computer-vision processing can evaluate images and return health labels, confidence values, and result summaries, which aligns with smart-greenhouse monitoring and image-based plant-health research [3, 4].

Layer	Role
Frontend / Vision Console (AH-110)	Operator interface for trays, plants, image history, devices, schedules, capture controls, and plant-health presentation.
Backend API / Application Services (AH-111)	Authenticated route handling, validation, orchestration, device commands, image ingest, and response shaping.
Database / Data Layer (AH-112)	Structured records for trays, plants, captures, devices, commands, schedules, predictions, reports, and monitoring events.
Storage File Management (AH-113)	Original and processed image files linked to capture records and served through application-mediated retrieval.
Hardware / Camera Layer (AH-114)	Raspberry Pi edge host, rack-mounted XY camera carriage, camera module, plant rack, movement limits, and capture wiring.
Optional Computer Vision	Image analysis adapters for plant-health classification, confidence reporting, and report generation.

Table 1: AgriHome system layers.

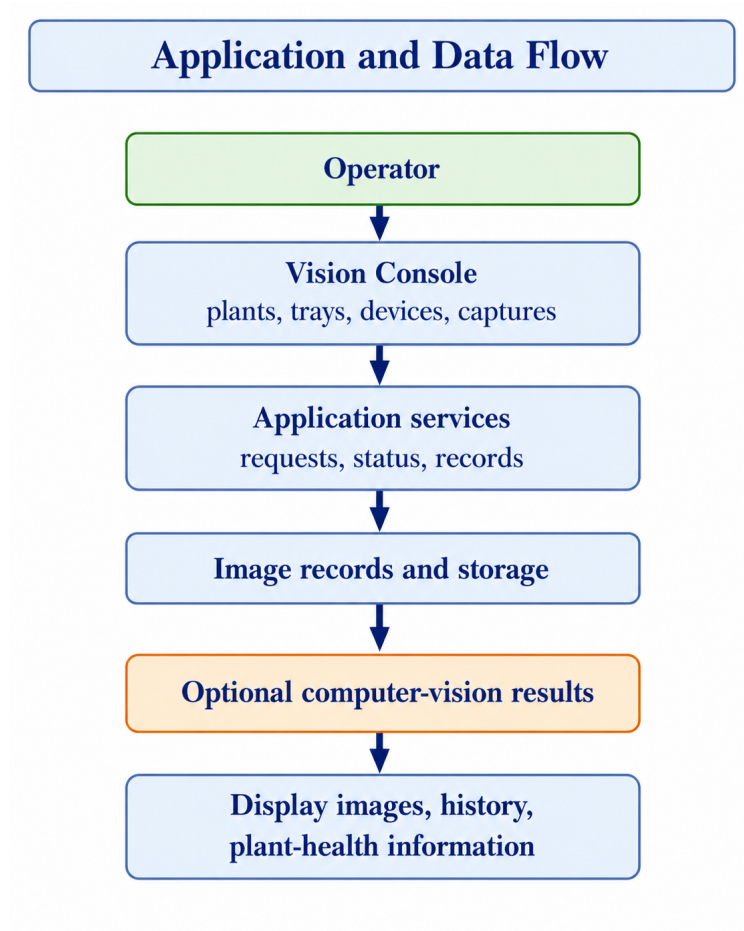


Figure 1: System Overview

2 AH-110: FRONTEND LAYER SUBSYSTEMS

2.1 LAYER PURPOSE

The Frontend Layer presents authenticated operator workflows for monitoring trays and plants, reviewing image history, managing schedules and meshes, viewing device status, requesting captures, and reading plant-health results. The user-facing design keeps the core workflows visible without requiring the operator to understand the database, edge-agent protocol, or hardware-control details.

2.2 LAYER HARDWARE

Any modern laptop, desktop, tablet, or phone with a contemporary browser can operate the Vision Console. No dedicated client GPU is required. A larger screen improves tray comparison and image-history review, while mobile screens support field checks and quick capture requests.

2.3 LAYER OPERATING SYSTEM

The Frontend Layer is OS-agnostic because it runs in a browser or installable web-app shell. Supported operator environments include Windows, macOS, Linux, iOS, and Android browsers with standard networking and media-display capabilities.

2.4 LAYER SOFTWARE DEPENDENCIES

Dependency Type	Role
Browser application shell	Routes the operator through Overview, Trays, Plants, Mesh, Schedule, Devices, Settings, and capture controls.
Typed component code	Keeps UI data contracts consistent across forms, tables, image cards, and status views.
Responsive styling utilities	Provide desktop sidebar navigation, mobile navigation, safe-area spacing, status badges, and image-grid layouts.
Session client	Maintains the human operator session and forwards authenticated requests to application services.
Charting and status widgets	Render plant-health trends, monitoring history, device freshness, and capture activity.

Table 2: Frontend layer software dependencies.

2.5 DASHBOARD OVERVIEW SUBSYSTEM

Aspect	Design
Route / View	Overview dashboard with protected access and operator-specific state.
Purpose	Summarize trays, plant status, latest captures, monitoring events, schedule activity, and edge-device availability.
Data	Tray summaries, plant counts, health status, recent captures, stale-device flags, and monitoring events.
Failure behavior	Empty states are shown when no trays or captures exist; session failure redirects to sign-in; stale devices are marked without blocking the dashboard.

Table 3: Dashboard Overview subsystem design details

2.6 IMAGE FEED DISPLAY SUBSYSTEM

The image feed display subsystem shows the latest plant and tray imagery from capture records. Tray views show recent rack captures, plant views show the most recent image associated with a plant, and image history views allow operators to compare captures over time.

Aspect	Design
Purpose	Provide the visual record used by operators to inspect plant condition and capture quality.
Source records	Capture metadata, plant references, tray references, capture time, movement target, and media URL.
Media retrieval	Application-mediated image route returns stored bytes only after validating access.
UI behavior	Loading, missing-image, failed-image, and latest-image states are distinct so operators know whether capture or retrieval failed.

Table 4: Image Feed Display subsystem design details

2.7 PLANT HEALTH RESULT UI SUBSYSTEM

Plant and tray pages present health status, diagnosis text, confidence values, prediction history, reports, and monitoring events. The UI treats computer-vision output as assistive information rather than absolute truth. Result cards show the image context, timestamp, model confidence where available, and notes when the model cannot provide a reliable conclusion.

2.8 MONITORING HISTORY SUBSYSTEM

Monitoring history is shown per plant and per tray. Events include capture completion, failed capture attempts, stale devices, schedule runs, operator notes, and plant-health updates. Historical captures remain browsable subject to storage retention.

2.9 ALERTS AND NOTIFICATIONS UI SUBSYSTEM

The alert UI surfaces warnings and critical conditions as in-console badges and event rows. Device stale marking appears on the Devices view and in tray context. Remote push or email notifications are outside the prototype scope, but the event model leaves room for those delivery channels.

2.10 RESPONSIVE NAVIGATION AND LAYOUT SUBSYSTEM

The application shell provides desktop navigation, mobile navigation, consistent page headings, contextual actions, and capture controls. Main operator workspaces are Overview, Trays, Plants, Mesh, Schedule, Devices, and Settings. Buttons and forms preserve spacing so labels do not cross margins on small screens.

2.11 DEVICE MANAGEMENT AND CAPTURE CONTROL SUBSYSTEM

The Devices view and tray capture panel expose edge-device status, linked tray information, last heartbeat, last capture result, and immediate Take Picture actions. A capture request identifies the target tray or plant position, sends it to application services, and displays command progress until the edge device completes or reports failure.

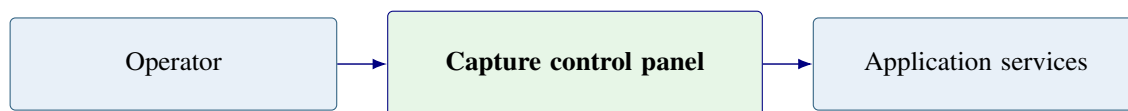


Figure 2: Capture control UI subsystem: operator requests become device capture commands.

3 AH-111: BACKEND API LAYER SUBSYSTEMS

3.1 LAYER PURPOSE

The Backend API Layer authenticates operators and devices, validates request payloads, orchestrates domain services, persists data, manages device commands, accepts image ingest, and optionally requests computer-vision analysis. It is the primary boundary between human UI workflows, durable storage, and edge hardware.

3.2 LAYER HARDWARE

Application services run on a developer machine, lab server, or deployment host with network access to the relational database, media storage, and Raspberry Pi edge devices. The design does not require the application host to control hardware directly; hardware actions are routed through the edge device.

3.3 LAYER OPERATING SYSTEM

The application host can run on a mainstream server operating system. The design assumes network access, filesystem access for media storage, and a runtime capable of serving HTTP requests, processing multipart uploads, and making outbound requests to optional model services.

3.4 LAYER SOFTWARE DEPENDENCIES

The Backend API Layer depends on route-handler infrastructure, a relational database client, session verification, device-key verification, multipart image parsing, file-storage helpers, command orchestration services, and optional HTTP clients for computer-vision adapters. Public request behavior follows standard HTTP semantics, and identity boundaries follow least-trust service-design principles [5, 6].

3.5 AUTHENTICATION AND USER MANAGEMENT API

Human identity and device identity are separated. Human operators authenticate through the normal operator session flow. Raspberry Pi edge devices authenticate through provisioned device credentials so they can register, heartbeat, claim commands, and ingest captures without becoming human users.

- Session creation establishes an operator context used for tray, plant, capture, schedule, and device-management actions.
- Logout clears the operator session.
- Protected routes require a verified operator before returning owner-scoped resources.
- Device routes require a valid device key and apply only to the provisioned device record.

3.6 PLANT AND TRAY MANAGEMENT API

Plant and tray services support creation, updates, deletion, status display, plant-to-tray association, mesh grouping, schedule association, capture history, and reports. The API validates that a requesting operator owns the relevant tray or plant before returning or mutating records.

3.7 IMAGE UPLOAD AND RETRIEVAL API

Image ingest accepts operator uploads and edge-device multipart uploads. Stored media receives a generated storage reference and a capture record. Retrieval streams image bytes through an application-mediated route so access control and path allowlisting remain centralized.

3.8 EDGE DEVICE API

AgriHome treats edge capture as a first-class workflow. Edge APIs support provisioning, heartbeat, command polling, capture-plan retrieval, pose retrieval, image ingest, and status reporting.

- Registration creates or links an edge device to a tray and returns a one-time device credential.
- Heartbeat updates online status, reports local device state, and may claim pending commands.
- Commands tell the edge device to capture now, synchronize poses, report status, or recover.
- Ingest uploads image bytes and metadata after the camera reaches the requested target.
- Capture-plan and pose endpoints define rack positions for repeatable X-axis and Y-axis movement.

3.9 EDGE SERVICE MAPPING

Capability	Service Responsibility	Primary Records or APIs
Edge registration	Create or recover a provisioned edge device, link tray context, and issue a device credential.	edge_devices, tray_systems, registration API.
Heartbeat and stale marking	Update last-seen status, report local state, and mark offline devices when stale.	edge_devices, heartbeat API, device listing API.
Capture commands	Queue, claim, complete, fail, and audit hardware commands.	edge_device_commands, operator device actions.
Image ingest	Accept image bytes, validate metadata, write media, and insert capture records.	camera_captures, storage routes, ingest API.
Plant attachment	Associate a capture with a plant, slot, or pose target when available.	plants, capture_poses, camera_captures.
Vision hook	Optionally analyze a saved capture and normalize the output.	prediction_results, plant_reports, monitoring_events.
Postprocess	Update image status, plant last-image fields, and monitoring events after ingest.	camera_captures, plants, monitoring_events.

Table 5: Edge capabilities mapped to services and records.

3.10 PLANT HEALTH ANALYSIS API

Plant-health analysis services accept an image reference and optional plant context, call the selected model adapter when configured, normalize the result, and persist predictions or reports. When model analysis is unavailable, the UI still preserves captured images and indicates that analysis is pending or unavailable.

3.11 ALERTS API

Alerts are represented as monitoring events and status flags. Device stale marking, failed capture attempts, plant-health warnings, schedule misses, and image-ingest failures can all create operator-visible events. Dedicated push notification delivery remains future scope.

4 AH-112: DATABASE / DATA LAYER SUBSYSTEMS

4.1 LAYER PURPOSE

The Data Layer provides durable structured storage for AgriHome domain entities, edge lifecycle state, capture history, analysis outputs, and monitoring events. It separates structured records from media bytes so capture metadata can be queried without loading image files.

4.2 LAYER HARDWARE

The Data Layer runs on a relational database server, local database instance, lab VM, or managed database service. It must support persistent storage, backups, indexing, concurrent reads, and transactional writes.

4.3 LAYER OPERATING SYSTEM

The database server can run on any supported server operating system. Application services access it over a configured database connection and do not expose direct database access to browsers or edge devices.

4.4 LAYER SOFTWARE DEPENDENCIES

The Data Layer depends on a relational database engine, schema migration scripts, connection pooling or equivalent connection management, and service-layer validation. Migrations define edge devices, command queues, image-ingest metadata, and capture-pose records.

4.5 DATABASE SCHEMA SUMMARY

The current design does not rely on fictional generic tables such as users, images, sensor_readings, or health_results. Human identity is represented through the operator session, while relational records store AgriHome domain state keyed to trays, plants, devices, captures, and ownership.

Table or Record Group	Role
tray_systems	Tray metadata, rack grouping, health status, crop context, optional edge-device association, and last capture fields.
plants	Plant records in trays, slot or row/column position, cultivar, diagnosis fields, health status, and latest image references.
camera_captures	Capture metadata, image URL, capture status, plant/tray association, capture source, command ID, and pose fields.
edge_devices	Edge-device identity, credential hash, status, last heartbeat, travel limits, linked tray, and revocation state.
edge_device_commands	Pending, claimed, completed, and failed device commands with payloads and result data.
capture_pose_sequences / capture_poses	Ordered X-axis and Y-axis target plans for multi-plant or multi-position capture.
capture_schedules	Automatic capture interval, next run time, linked tray/device destination, and last-run metadata.
prediction_results / plant_reports / monitoring_events	Computer-vision outputs, structured reports, and operator-visible event stream.
mesh_networks	Groupings of trays or racks for higher-level organization.
feedback_ingest / user_preferences	Optional feedback samples, preferences, and consent-related settings.

Table 6: AgriHome relational record groups.

4.6 TRAY_SYSTEMS TABLE

The tray_systems table stores tray identity, owner context, name, zone, crop, plant counts, health status, linked device information, last capture time, and summary fields. It is the anchor for tray views, schedules, rack grouping, and edge-device linking.

4.7 PLANTS TABLE

The plants table stores plant identity, tray foreign key, mesh membership, display name, cultivar, slot or position fields, health status, latest diagnosis, optional notes, last image URL, and last image timestamp. It supports both manual plant creation and photo-assisted plant creation.

4.8 CAMERA_CAPTURES TABLE

The camera_captures table stores image metadata rather than raw image bytes. It includes image URL, captured time, source, status, optional plant ID, tray ID, command ID, X-axis or Y-axis target information, pose order, and error details when capture fails.

4.9 EDGE_DEVICES TABLE

The edge_devices table stores edge identity, credential hash or prefix, status, last heartbeat, linked tray, hostname or serial information, configured movement limits, and revocation state. Device records allow operators to see whether a Raspberry Pi is online, stale, revoked, or pending setup.

4.10 EDGE_DEVICE_COMMANDS TABLE

The command queue records actions such as capture_now, sync_poses, report_status, and recover. Each command stores payload data, command status, claim time, completion time, result fields, and failure message if the edge device cannot complete the operation.

4.11 CAPTURE_POSE_SEQUENCES / CAPTURE_POSES

Pose sequence records define ordered capture plans for a tray or rack. A sequence groups individual poses, while each pose may include target row, target column, X-axis position, Y-axis position, dwell time, plant ID, or slot binding. This enables repeatable individual plant capture.

4.12 CAPTURE_SCHEDULES TABLE

The capture_schedules table defines interval-based or operator-defined capture plans. Records store cadence, next run time, last run time, destination, linked tray/device, and enabled state.

4.13 PREDICTION_RESULTS / PLANT_REPORTS / MONITORING_EVENTS

Prediction results store model labels, confidence values, image linkage, and model metadata. Plant reports convert predictions or operator notes into structured summaries. Monitoring events form the alert and activity stream used by dashboards and history views.

4.14 INDEXING, RETENTION, AND VALIDATION STRATEGY

Indexes support owner-scoped reads, recent captures, pending commands, edge status, plant history, and pose ordering. Application services validate ownership, foreign keys, file references, device credentials, and allowed state transitions before writing. Image retention is operationally governed by storage capacity and project handoff policy.

5 AH-113: STORAGE FILE MANAGEMENT SYSTEMS

5.1 LAYER PURPOSE

The Storage File Management Layer stores binary imagery referenced by relational capture records. It keeps large media files out of source control and out of ordinary database rows while preserving stable references for the Vision Console.

5.2 LAYER HARDWARE

Storage uses local disk, mounted volume, or server-managed storage available to the application host. The prototype assumes enough capacity for original captures, processed images, temporary uploads, and test artifacts.

5.3 LAYER OPERATING SYSTEM

The design depends on ordinary host filesystem semantics. Access permissions must prevent raw browsing of storage folders and must restrict write access to the application process.

5.4 LAYER SOFTWARE DEPENDENCIES

Storage helpers resolve storage root, originals, processed, and temporary directories. Upload services validate type and size, write image bytes, create generated file names, and return a media reference for capture records.

5.5 FILE MANAGEMENT SUBSYSTEMS

Subsystem	Processing
Image Upload	Validate file type and size, write original image, and insert or update camera_captures metadata.
Folder Organization	Use generated names under logical originals, processed, and temp folders.
Image Retrieval	Stream authorized bytes through an application-mediated image route.
Cleanup / Archiving	Support manual cleanup for prototype storage and future cold-archive policy.
Security	Require operator session for UI uploads and device key for edge ingest; protect storage folders at the host level.

Table 7: Storage subsystem summary.

5.6 IMAGE UPLOAD SUBSYSTEM

The image upload subsystem accepts operator uploads and edge-device multipart ingest. It validates file type, validates expected metadata, stores the image file, creates a stable media URL or storage key, and inserts the matching capture record.

5.7 FILE NAMING AND FOLDER ORGANIZATION SUBSYSTEM

File names are generated rather than trusted from users or devices. The system groups images into logical storage folders and stores database references to those generated paths.

5.8 IMAGE RETRIEVAL SUBSYSTEM

The image retrieval subsystem streams bytes through application services after checking authorization and path allowlisting. The Vision Console receives displayable image resources without direct filesystem access.

5.9 IMAGE CLEANUP AND ARCHIVING SUBSYSTEM

Prototype cleanup is an operations activity. Future archive jobs may move older originals or processed images to cold storage while keeping relational metadata in place.

5.10 SECURITY AND ACCESS CONTROL SUBSYSTEM

Storage is not publicly browsable. Operator uploads require an authenticated session, edge ingest requires valid device credentials, and host OS permissions must restrict direct filesystem access.

6 AH-114: HARDWARE / CAMERA LAYER SUBSYSTEMS

6.1 LAYER PURPOSE

The Hardware / Camera Layer captures individual plant images on a rack or shelf system and delivers them to AgriHome with minimal operator effort. Its primary responsibility is repeatable visual monitoring, not autonomous watering, dosing, or treatment.

6.2 LAYER HARDWARE

- Raspberry Pi-class edge computer or similar local controller.
- Camera or webcam module attached to an XY carriage on the plant rack or shelf.
- X-axis movement mechanism for horizontal travel across plant columns.
- Y-axis movement mechanism for vertical travel between plant rows or rack levels.
- Plant rack, tray materials, camera fixture, mounting hardware, travel-limit hardware, and lighting.
- Power and networking for the edge device, camera module, and motion hardware.

Component	Role
Raspberry Pi edge device	Runs the local agent, stores device configuration, claims commands, reports heartbeat, and uploads capture results.
Camera module	Captures plant images at selected rack coordinates.
XY camera carriage	Moves the camera horizontally and vertically to target individual plants.
Plant rack / tray fixture	Holds plants at repeatable positions and provides physical reference for coordinate mapping.
Lighting and mounting	Improves image consistency and keeps camera framing stable.

Table 8: Hardware prototype component summary.

6.3 LAYER OPERATING SYSTEM

The edge host runs a lightweight Linux operating system with camera access, network access, local configuration, and the AgriHome edge agent. The agent must restart reliably after power loss or network interruption.

6.4 LAYER SOFTWARE DEPENDENCIES

Hardware software includes a local edge agent, camera capture library or command interface, motion-control driver or controller interface, HTTP client for application communication, local configuration file, logging, retry handling, and calibration data for X-axis and Y-axis travel limits.

6.5 CAMERA CAPTURE SUBSYSTEM

The camera capture subsystem receives a target plant or pose, waits for the carriage to reach the requested position, captures an image, validates that an image file exists, packages metadata, and returns the file for ingest. Image-quality factors include lighting, focus, plant placement, vibration, and camera angle.

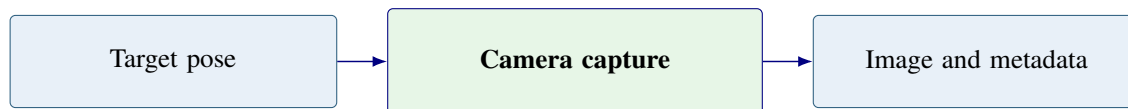


Figure 3: Camera capture subsystem: target pose becomes an ingest-ready plant image.

6.6 PLANT RACK POSITIONING SUBSYSTEM

The positioning subsystem maps tray row and column targets into X-axis and Y-axis movement. Capture poses define ordered movement targets and dwell timing. Travel-limit checks prevent unsafe motion. Calibration records allow the same plant to be recaptured consistently over time.

6.7 DEVICE COMMUNICATION SUBSYSTEM

The edge agent communicates with AgriHome through registration, heartbeat, command polling, capture-plan retrieval, pose retrieval, image ingest, and status reporting. The Vision Console never talks directly to motion hardware; it requests capture through application services.

6.8 POWER AND RELIABILITY SUBSYSTEM

Power reliability depends on stable edge-device power, safe cable routing, protected camera wiring, and reliable network access. Heartbeat stale windows mark offline devices. Credential rotation and device revocation support recovery after replacement or compromise.

6.9 HARDWARE LAYER LIMITATIONS

- Camera capture quality depends on lighting, focus, plant placement, and carriage stability.
- XY movement requires calibration before accurate individual-plant capture.
- Network interruption can delay command completion or image upload.
- The prototype does not include autonomous plant-care actuation.
- Environmental sensors remain optional relative to vision-based capture.

7 AH-115: SYSTEM OVERVIEW AND ARCHITECTURE DIAGRAM

7.1 ARCHITECTURE EXPLANATION

AgriHome is a layered web application with an edge capture extension. The Vision Console does not access storage or relational data directly. Application services mediate all operator and device requests. Device credentials are distinct from human operator sessions. Optional computer-vision analysis sits behind a stable processing boundary so the rest of the system can function even when analysis is unavailable.

7.2 ARCHITECTURE DIAGRAM

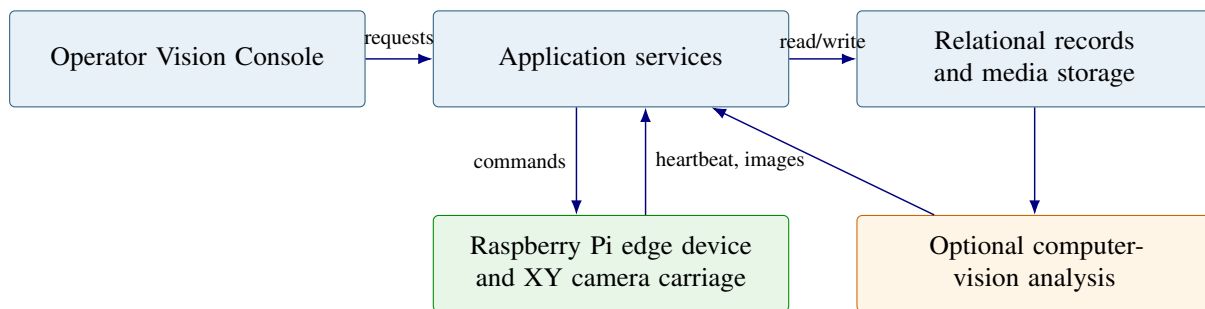


Figure 4: AgriHome runtime detail flow for operator actions, edge captures, storage, and optional analysis.

7.3 ARCHITECTURE DATA FLOW

1. An operator signs in and opens a dashboard, tray, plant, device, schedule, or capture-control view.
2. The Vision Console requests records from application services.
3. Application services validate the operator session, enforce ownership, and read or write relational records.
4. A Take Picture request creates or dispatches a device command with target tray, plant, or pose information.
5. The Raspberry Pi edge device heartbeats, claims the command, moves the rack-mounted XY camera carriage, captures an image, and uploads bytes plus metadata.
6. Storage services save the image, the data layer records capture metadata, and optional computer-vision services analyze the capture.
7. The Vision Console displays the updated image, command status, device status, and plant-health information.

7.4 INTERFACE SUMMARY

ID	Interface	Inputs	Outputs
I-01	Operator session to application services	Sign-in, logout, or session refresh request	Verified operator context or redirect state
I-02	Vision Console to tray and plant APIs	Tray, plant, mesh, or schedule request	Owner-scoped records and validation errors
I-03	Vision Console to device API	Capture, link, revoke, rotate, or status request	Device action result and command state
I-04	Edge device to registration API	Provisioning data and hardware fingerprint	Device record and device credential
I-05	Edge device to heartbeat API	Device key, status, local state	Online status and optional command claim
I-06	Edge device to pose or capture-plan API	Device key and tray context	Ordered target positions and movement limits
I-07	Edge device to ingest API	Image bytes, capture metadata, command result	Capture record and media reference
I-08	Application services to media storage	Validated image file and metadata	Stored file path or image URL
I-09	Application services to computer-vision adapter	Image reference and plant context	Normalized labels, confidence values, and report data
I-10	Vision Console to file retrieval route	Authorized image request	Displayable image bytes

Table 9: Backend API endpoint summary for selected interfaces

7.5 DESIGN RATIONALE

The design keeps operator workflows, application services, persistent records, media files, processing adapters, and physical hardware in separate layers. This makes testing easier, reduces the chance that a hardware failure corrupts domain records, and allows the edge device to recover independently. Storing image bytes separately from structured records keeps database operations fast while preserving a complete capture history.

The rack-mounted XY camera design supports repeatable imaging because each plant can be captured from a known target position. Repeatable images are important for comparing plant-health changes over time and for producing more consistent inputs to optional computer-vision analysis and phenotyping workflows [7].

7.6 CURRENT IMPLEMENTATION CONNECTION

Implementation anchors include Vision Console components, application route handlers, edge-device services, storage helpers, capture-pose services, command-queue services, relational schema migrations, and the Raspberry Pi edge agent. These are implementation details behind the design boundaries described above.

8 AH-116: FINAL REVIEW AND SUBMIT

8.1 FINAL REVIEW CHECKLIST

- Cover page, roster, revision history, footer, contents, list of figures, and list of tables follow the UTA-style document format.
- AH-109 through AH-116 sections are complete and aligned with the current AgriHome system.
- Frontend design includes dashboard, trays, plants, image history, devices, schedules, alerts, and capture-control workflows.
- Backend design documents application services, operator sessions, device credentials, capture commands, image ingest, retrieval, and optional analysis.
- Hardware section covers Raspberry Pi edge device, rack-mounted XY camera carriage, X-axis and Y-axis movement, plant rack fixture, and capture limitations.
- Interface table matches the current operator, device, storage, and analysis boundaries.
- Cross-check confirms that the document consistently describes the rack-mounted XY camera system and current AgriHome subsystem boundaries.

8.2 REVISION HISTORY TABLE

The front-matter revision history is the authoritative revision record for this DDS. Revision 2.0 records the comprehensive detailed design revision dated 07.26.2026.

8.3 LIST OF FIGURES / TABLES

The list of figures and list of tables are generated from the LaTeX source and updated through repeated compilation.

8.4 APPENDIX SUGGESTIONS

Suggested supporting material includes calibration sheets, example edge-device configuration, capture-quality checklist, sample command lifecycle logs, database schema snapshots, and screenshots of successful Vision Console workflows.

9 APPENDIX A: SUGGESTED SUPPORTING MATERIAL

- Rack-coordinate calibration sheet showing target plant positions, X-axis movement range, Y-axis movement range, and safe travel limits.
- Edge-device setup checklist covering provisioning, heartbeat, command claim, capture, ingest, and recovery.
- Capture-quality checklist covering lighting, focus, framing, vibration, image resolution, and plant visibility.
- Sample capture record showing tray ID, plant ID, command ID, capture time, pose target, image URL, status, and optional analysis result.
- Demonstration script covering add tray, add plant, link device, take picture, view image history, and review plant-health result.

10 APPENDIX B: SUBSYSTEM TO MODULE / TABLE MAP

DDS Subsystem	Implementation Area	Primary Records
Dashboard / navigation	Vision Console shell and dashboard routes	tray_systems, monitoring_events, camera_captures
Image feed	Image cards, file retrieval route, camera capture services	camera_captures, plants
Plant health UI	Plant/tray pages, prediction views, report views	plants, prediction_results, plant_reports
Edge registration	Device provisioning service and registration route	edge_devices, tray_systems
Heartbeat	Device heartbeat service and stale-device checks	edge_devices
Capture commands	Device action service and command queue	edge_device_commands
Ingest / capture storage	Edge ingest service, media storage helpers	camera_captures and image files
Plant attachment	Capture-to-plant association service	plants, camera_captures, capture_poses
Vision hook	Optional processing adapter and result normalizer	prediction_results, plant_reports, monitoring_events
Poses	Capture-pose management service	capture_pose_sequences, capture_poses
Schedules	Schedule management and capture-plan service	capture_schedules

Table 10: DDS subsystems mapped to implementation areas and records.

11 REFERENCES

REFERENCES

- [1] ISO/IEC/IEEE, *ISO/IEC/IEEE 42010:2022 Systems and Software Engineering – Architecture Description*, International Organization for Standardization and IEEE, 2022.
- [2] IEEE Computer Society, *IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions*, IEEE, 2009.
- [3] C. Bersani, C. Ruggiero, R. Sacile, A. Soussi, and E. Zero, “Internet of things approaches for monitoring and control of smart greenhouses in Industry 4.0,” *Energies*, vol. 15, no. 10, p. 3834, 2022.
- [4] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, no. 1419, 2016.
- [5] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” Internet Engineering Task Force, Tech. Rep. RFC 9110, 2022.
- [6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-207, 2020.
- [7] J. F. Humplík, D. Lazár, A. Husíčková, and L. Spíchal, “Automated phenotyping of plant shoots using imaging methods for analysis of plant stress responses: A review,” *Plant Methods*, vol. 11, p. 29, 2015.